Recolección de residuos

Ignacio Mastrangelo Renzo Portela

Informe de proyecto — Machine Learning y Grafos

Índice

1 El problema **2**
1.1 Tecnologías y piezas . . . 2

2 Cómo evolucionó el trabajo: decisiones, dificultades y correcciones **2**

3 Datos y definición del modelo **2**
3.1 Procedencia, alcance y qué significa "real" . . . 2
3.2 Qué CSV usa el experimento y cuál usa la aplicación . . . 3
3.3 De las nueve columnas a X e y . . . 3

4 Desarrollo, comparación y lectura del modelo **4**
4.1 Protocolo para limitar sesgos y fuga . . . 4
4.2 Resultados del piloto y errores . . . 5
4.3 Prueba con más datos (recorte medio) . . . 6
4.4 Interpretación y alcance de las métricas . . . 6
4.5 La dificultad de los datos sintéticos . . . 7
4.6 Hallazgo del árbol: tramos sin contenedores en media . . . 7
4.7 Limitaciones principales . . . 8
4.8 Propiedad observada al cierre del desarrollo del modelo . . . 8

5 Aplicación, API y uso de inteligencia artificial **8**
5.1 Integración y contratos actuales . . . 8
5.2 La aplicación en funcionamiento . . . 9
5.3 Declaración de uso de IA . . . 10

6 Grafos: del conjunto de destinos al circuito **10**
6.1 Cómo se conectan el modelo y el grafo . . . 10
6.2 Qué implementa el planificador . . . 11
6.3 Integración y límites prácticos . . . 11

1 El problema

Dada una red vial con contenedores, el objetivo es (1) clasificar la prioridad de recolección de cada tramo y (2) construir un circuito que cubra los tramos prioritarios. El flujo implementado es: datos → modelo → prioridades → selección de aristas → cartero chino/rural → API/mapa.

1.1 Tecnologías y piezas

El análisis de datos, los clasificadores y las figuras se apoyan en Python, pandas, NumPy, scikit-learn y Matplotlib. El adaptador de rutas utiliza NetworkX para el grafo, dijkstra y el emparejamiento; Hierholzer es implementacion propia. Las geometrías se trabajan en WGS84 y se integran en una aplicación Next.js/React con Leaflet. FastAPI expone los contratos de datos, predicción, explicación y planificación. Los archivos de dependencias son `requirements.txt` y `backend/requirements.txt`; el contrato vigente está descrito en `docs/contracts/api-v1.md` y la implementación HTTP es `backend/src/collection/transport/api_v1.py`.

2 Cómo evolucionó el trabajo: decisiones, dificultades y correcciones

1. **Definir una unidad geográfica utilizable.** Se partió de ejes viales y coordenadas de contenedores, y se construyeron nodos y tramos con IDs para unir geometría, atributos y rutas. El recorte piloto permitió probar el flujo sin confundir un ejercicio de curso con un sistema listo para procesar toda la ciudad.
2. **Recalibrar la etiqueta y después enriquecerla.** La regla inicial (issue 8) daba casi todas las filas como baja: la comparación histórica registra 99.7 por ciento bajas. Primero se recalibró la regla sin quejas (issue 9); ese resultado ya recalibrado fue 129 bajas, 125 medias y 43 altas en el piloto. Después, la regla 18 añadió quejas sintéticas y produjo 129 bajas, 93 medias y 75 altas en 297 filas. La prioridad nunca provino del municipio: son etiquetas construidas para el ejercicio.
3. **Evitar que la evaluación mire información futura.** Se pasó a pipelines sin ajustar dentro de cada fold; la elección de modelo se hace con validación, y el conjunto de test se reserva para una única confirmación del elegido.
4. **Resolver una discrepancia en los resultados comparativos.** En el piloto, árbol y SVC RBF empataron en validación. Se eligió el árbol por interpretabilidad. En una auditoría metodológica separada, `docs/comparacion-evaluacion/CONCLUSION.md` documenta otro protocolo (CV sobre train completo) que escoge KNN; no se mezclan esos resultados con el pipeline de producción.
5. **Aprender del aumento de escala.** Al preparar el recorte medio, el gate de IDs detectó `tramo_id` duplicados. Se corrigió mediante un contador de ocurrencias global por eje.
6. **Integrar el modelo sin entrenarlo en cada petición.** La API carga un pipeline ya ajustado (`backend/artifacts/arbol-montevideo.joblib` por defecto).

3 Datos y definición del modelo

3.1 Procedencia, alcance y qué significa "real"

La red vial deriva de OpenStreetMap/Geofabrik; la fuente y huella del extracto están consignadas en `docs/dataset-recoleccion.md`. Las coordenadas de contenedores proceden del conjunto de Toyos y Vallcorba, *Montevideo Garbage Containers*, Figshare, DOI https://doi.org/10.6084/m9.figshare.17126738.v1. Para los levantes se usó el atributo `daysWithoutLift` de la

instantánea de contenedores, agregando por tramo el máximo de sus valores. El proyecto documenta una foto T0 del 23 de octubre de 2026 en `dataset/09_levantes/README.md`; por ello no hay una serie temporal de entrenamiento.

La mezcla de procedencias es importante: geometría y contenedores son datos reales derivados; días procede de registros de levantes de una foto; `tipo_zona` se asignó de forma simulada (semilla 42); las quejas se generan en forma sintética y anónima por contenedor representado; y `prioridad_recoleccion` es una etiqueta sintética, a partir de la regla (fórmula 18). El conjunto de Windsor citado en la documentación aportó estadísticas agregadas de referencia al diseñar la simulación; el sorteo implementado de quejas es uniforme entero en $[0, 3]$ por contenedor y no usa esos estadísticos en el código. Sus filas no se unen a Montevideo.

3.2 Qué CSV usa el experimento y cuál usa la aplicación

La comparación reportada para el piloto usa como entrada física `dataset/salida/tramos.csv`: tiene **297 filas de datos y 9 columnas**. La red completa para la aplicación está en `dataset/salida-montevideo/tramos.csv`: tiene **46 234 filas y las mismas 9 columnas**. Los números del experimento piloto no deben leerse como una evaluación de las 46 234 filas. El artefacto servido por defecto es `backend/artifacts/arbol-montevideo.joblib` (árbol ajustado en la partición train del CSV de ciudad, $N_{\text{train}} = 27\,740$; no usa las 46 234 filas enteras); las métricas de este informe son del piloto.

`Columna física`	Significado y uso
`tramo_id`	ID estable del tramo. Trazabilidad y quejas reproducibles; no es predictor.
`nodo_origen`	Nodo inicial de la arista. Estructura del grafo, no predictor.
`nodo_destino`	Nodo final de la arista. Estructura del grafo, no predictor.
`longitud_m`	Longitud en metros. Costo del grafo; no entra al clasificador.
`nombre_calle`	Nombre descriptivo; no predictor.
`tipo_zona`	Residencial / comercial / industrial (simulada). Entra a X (one-hot).
`cantidad_contenedores`	Conteo real-derivado. Entra a X.
`dias_desde_ultima_recoleccion`	Máximo de `daysWithoutLift` en la foto T0. Entra a X.
`prioridad_recoleccion`	Etiqueta sintética generada por la regla número 18 en ambos CSV actuales.

Esas nueve columnas son las únicas del CSV en disco. `historico_quejas_vecinales` **no se persiste**: se agrega en runtime al preparar el dataset para ML y al servir el escenario (misma función determinista, seed 0, a partir de `tramo_id` y `cantidad_contenedores`). Así el modelo la usa como feature sin figurar en el archivo fuente.

Filas literales del CSV piloto:

ID	Origen	Destino	m	Calle	Zona	Cont.	Días	Etiqueta CSV
743359	N0045	N0062	127.07	QUEBRACHO	comercial	1	0	alta
743543	N0020	N0012	5.21	MANDIYU	residencial	0	0	baja
743988	N0131	N0120	116.62	RAMON MASINI	industrial	0	0	media

3.3 De las nueve columnas a X e y

El código `ml_preparation.py` define cuatro columnas de entrada; y se reconstruye con la regla número 18. La cuarta feature, `historico_quejas_vecinales`, se **inyecta en runtime** (no viaja

en el CSV): train y serving la regeneran con seed 0 por ID de tramo. Con esa semilla, la etiqueta recalculada coincide fila a fila con el CSV actual (297/297 y 46 234/46 234).

```
FEATURE_COLUMNS = (
    "cantidad_contenedores",
    "dias_desde_ultima_recoleccion",
    "tipo_zona",
    "historico_quejas_vecinales",
)
```

Fórmula #18: `score = 2*dias + contenedores + zona_w + 2*quejas`, con zona residencial=0, industrial=1, comercial=2; baja si score ≤ 0, media si $1 \leq score \leq 2$, alta si score > 2. El árbol aprende una regla que usa las mismas variables de las que se deriva y (circularidad de diseño declarada).

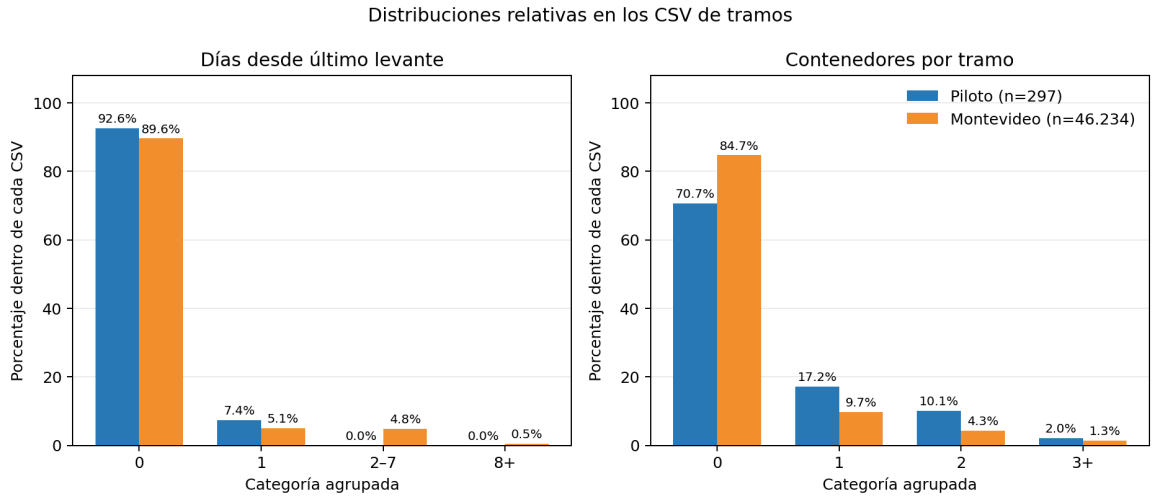


Figura 1: Distribuciones porcentuales normalizadas: piloto (n=297) y Montevideo (n=46 234). Comparación descriptiva de CSV, no evaluación del modelo.

4 Desarrollo, comparación y lectura del modelo

4.1 Protocolo para limitar sesgos y fuga

El piloto de 297 filas se dividió estratificadamente con semilla 0: 178 train, 59 validación y 60 test. En train, CV estratificada de cinco folds; grillas con F1 macro. Escaladores y codificadores dentro de cada pipeline/fold. Se escoge por validación; test una sola vez. Los cuatro clasificadores comparados (árbol, logística, KNN, SVC RBF) viven en `notebooks/10_entrenamiento_modelos.ipynb`; el backend solo declara y sirve el árbol.

```
# notebooks/10_entrenamiento_modelos.ipynb: grillas de la comparacion
"arbol": {"decisiontreeclassifier__max_depth": [None, 3, 5],
          "decisiontreeclassifier__min_samples_split": [2, 5]},
"logistica": {"logisticregression__C": [0.5, 1.0, 2.0]},
"knn": {"kneighborsclassifier__n_neighbors": [3, 5, 7]},
"svc": {"svc__C": [0.5, 1.0]},
# backend model_specs.py solo declara el arbol servido por la API
```

4.2 Resultados del piloto y errores

Modelo	CV F1 macro (media ± desv.)	Val. accuracy	Val. F1 macro	Test F1
Árbol	0.9932 ± 0.0136	1.0000	1.0000	1.0000
SVC RBF	0.9733 ± 0.0390	1.0000	1.0000	–
KNN	0.9743 ± 0.0315	0.9661	0.9680	–
Logística	0.9619 ± 0.0367	0.9322	0.9268	–
Baseline mayoría	–	0.4407	0.2039	–

El protocolo reservó test para el árbol elegido. En validación, árbol y SVC obtuvieron matrices diagonales perfectas. KNN confundió dos medias como baja; logística, dos medias→baja y dos altas→media.

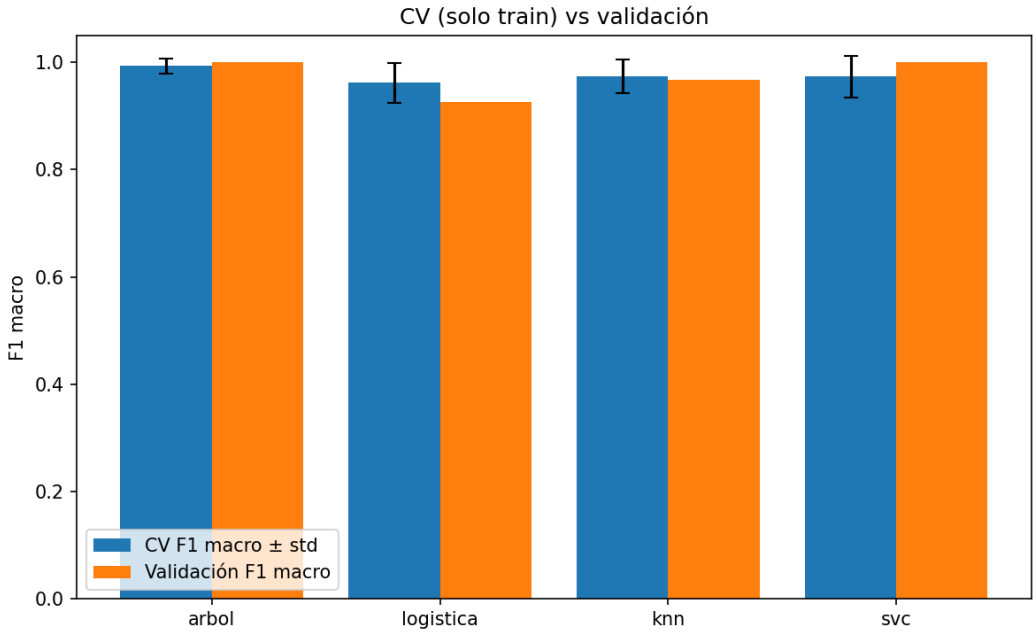


Figura 2: F1 macro de CV en train frente a validación (piloto). No es comparación sobre los 46 234 tramos de ciudad.

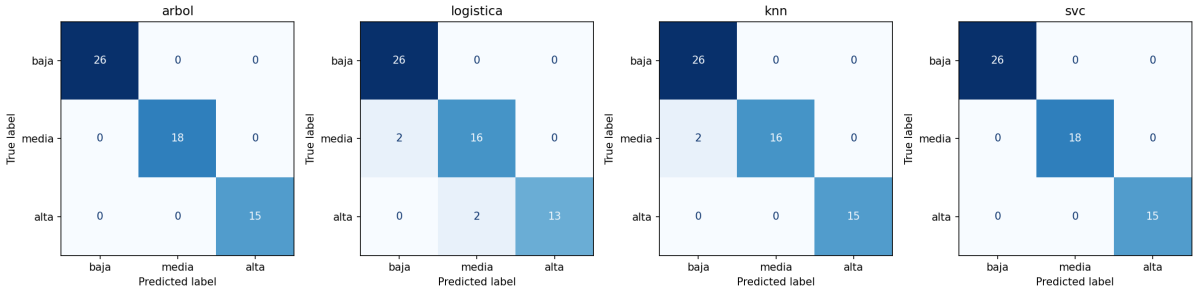


Figura 3: Matrices de confusión de validación (filas: clase real; columnas: predicha; orden baja, media, alta).

Precision y recall macro en validación (piloto, $n = 59$), tomados del contraste documentado en `docs/modelo-comparacion.md` y reproducidos en `notebooks/10_entrenamiento_modelos.ipynb`:

Modelo	Precision macro	Recall macro
Árbol	1.0000	1.0000
SVC RBF	1.0000	1.0000
KNN	0.9762	0.9630
Logística	0.9392	0.9185

El árbol (y SVC) no cometen falsos positivos ni falsos negativos en validación. KNN pierde recall en media (2 medias reales predichas como baja). Logística mezcla clases vecinas (medias↔altas). En recolección, un falso negativo media→baja es caro: el tramo puede quedar fuera del circuito si el filtro es media+alta.

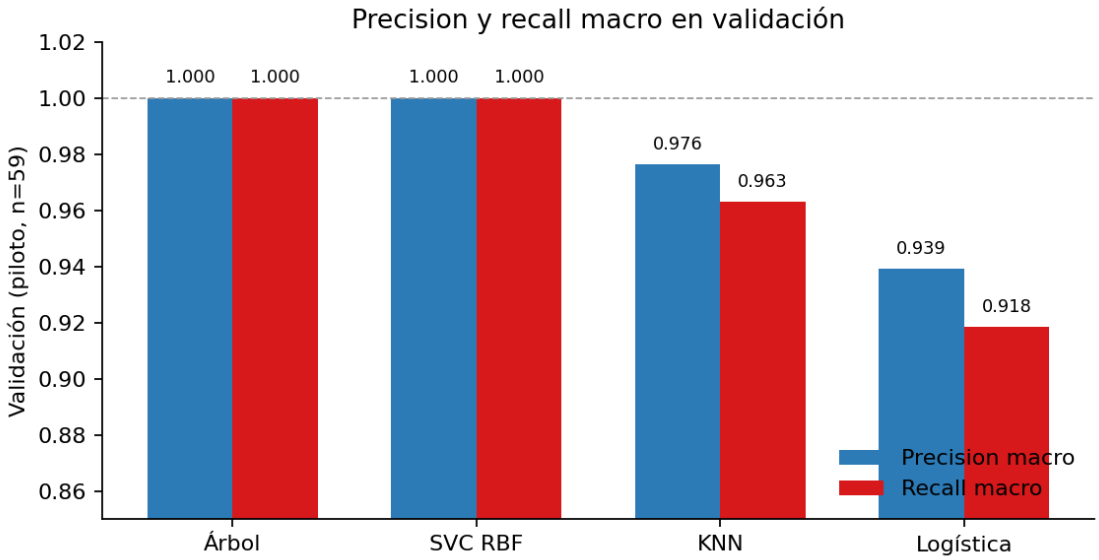


Figura 4: Precision y recall macro en validación (piloto). Fuente: contraste del notebook 10 / `docs/modelo-comparacion.md`.

Test de un solo uso (solo el árbol elegido, $n = 60$): accuracy, precision, recall y F1 macro = 1.0000. Eso confirma el ajuste a la etiqueta sintética en el hold-out; no valida operación real.

4.3 Prueba con más datos (recorte medio)

Para chequear si el piloto ($N = 297$) era demasiado chico, se repitió el mismo protocolo sobre un recorte medio de **1422 tramos** (split 853/284/285, misma semilla 0 y misma regla #18). Evidencia en `docs/modelo-comparacion.md` (sección de verificación de escala) y en la rama `experimento-recorte-medio`.

Resultado: el ranking se conserva (árbol≈SVC ≥ KNN > logística) y el elegido sigue siendo el árbol por el mismo desempate de interpretabilidad. Con más datos bajan los desvíos de CV (∼5×) y casi desaparecen los errores de KNN/logística en validación; el test del árbol sigue en 1.0. Conclusión: el piloto alcanza para decidir el modelo; el medio confirma que el resultado no era un accidente de N chico. Detalle numérico en `docs/modelo-comparacion.md`.

4.4 Interpretación y alcance de las métricas

El artefacto que sirve la API por defecto es `backend/artifacts/arbol-montevideo.joblib`: árbol ajustado solo en la partición train del CSV de ciudad ($N_{\text{train}} = 27\,740$) con `max_depth=None` y `min_samples_split=2`. Las métricas de esta sección (piloto y recorte medio) **no** validan ese artefacto de ciudad.

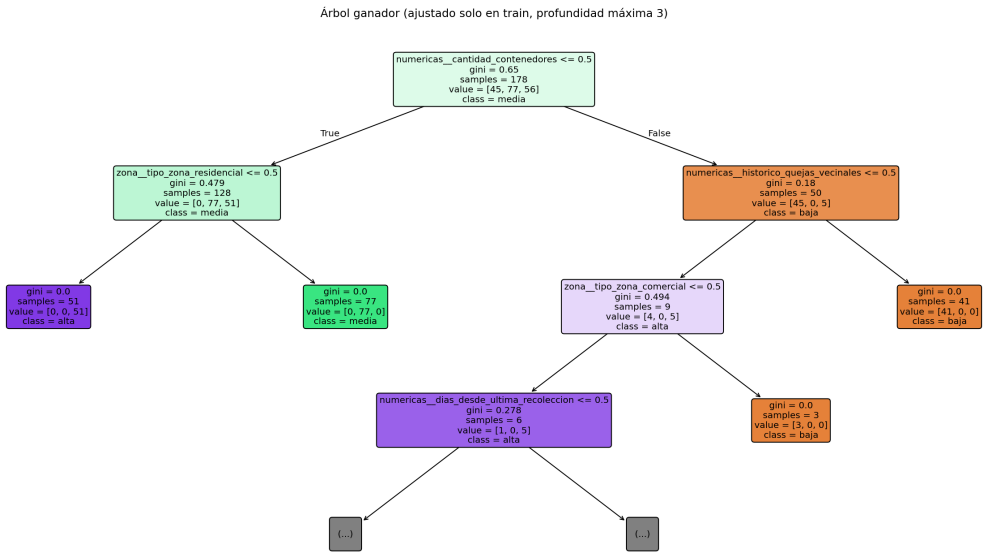


Figura 5: Árbol ganador (piloto), truncado a profundidad tres para lectura.

La exactitud y el F1 de 1.0 en test significan que, sobre las 60 filas reservadas del piloto, el árbol reprodujo la etiqueta sintética. No significan acierto operativo ni validación de ciudad. La revisión `docs/comparacion-evaluacion/CONCLUSION.md` selecciona KNN bajo otro protocolo; no se fusiona con este experimento.

4.5 La dificultad de los datos sintéticos

En el piloto ≈92,6% (275/297) tiene días cero; el peso doble de días está escasamente observado. Los CSV actuales llevan etiqueta #18 escrita por el builder; la preparación regenera quejas en memoria y recalcula y (coincide con el CSV).

4.6 Hallazgo del árbol: tramos sin contenedores en media

Durante la comparación (issue #10), al graficar el árbol ganador (Fig. 5, desde `notebooks/10_entrenamiento_mo` con el prompt "Revisa las respuestas generadas por el arbol y busca inconsistencias"), se detectó que ramas con `cantidad_contenedores == 0` caían en `media`. Ejemplo del piloto: tramo `743988` (RAMON MASINI, industrial, 0 cont., 0 días) etiquetado `media`. Fue el motivo para preferir el árbol frente al empate con SVC: solo el árbol deja ver este defecto.

Causa: no es error de aprendizaje —el árbol reproduce la regla con F1 1.0 porque $y = f(X)$ es determinista— sino diseño de la etiqueta #18 (`backend/src/collection/adapters/datasets/synthetic_comp` `score = 2*dias + contenedores + zona_w + 2*quejas` con `zona_w={residencial:0, industrial:1, comercial:2}` y umbrales `baja` $\leq$ `0, media` $\leq$ `2`. Con 0/0/0 las quejas son 0 por construcción, pero `zona_w` sola ya da score 1–2 → `media` para industrial/comercial; solo residencial 0/0/0 queda `baja`. Operativamente es incorrecto: sin contenedor no hay recolección.

Mitigación: la regla no se cambió (corregirla exige regenerar CSVs, re-correr `prepare+compare` e invalidar el test, procedimiento en `docs/modelo-comparacion.md`). Se contuvo aguas abajo: `NetworkxRoutePlanner._mandatory_ids` exige `container_count > 0`, por lo que un `media` sin contenedor nunca es obligatorio y solo sirve como tránsito. Detalle adicional en `ai.md`.

Cara inversa del mismo problema (`SCORE_LOW_MAX=0`): como todos los términos del score son ≥ 0, `baja` exige `dias=0, contenedores=0, zona=residencial, quejas=0` a la vez. En el piloto, las 129 filas `baja` tienen 0 contenedores (ninguna `baja` con contenedor; `alta`: 75/75 con contenedor; `media`: 81 sin + 12 con). Por eso `include=(baja)` produce selección vacía: `PlanRoute`

y `NetworkxRoutePlanner` lanzan `ImpossibleRoute` ("ningún tramo con contenedores en prioridades incluidas"). No es un bug del planificador, es consecuencia matemática del umbral: `baja` nunca es ruteable por sí sola.

4.7 Limitaciones principales

- No hay etiquetas oficiales ni evaluación operativa con rutas reales.
- El piloto es una foto temporal (297 tramos); no evalúa generalización temporal ni espacial.
- Tipo de zona y quejas son simulados.
- El buen score es esperable cuando $y = f(X)$ es determinista.
- El artefacto servido se ajusta en el train de ciudad ($N_{\text{train}} = 27\,740$), pero las métricas citadas aquí son del piloto.
- `baja` implica 0 contenedores por `SCORE_LOW_MAX=0`; un recorrido solo-`baja` es selección vacía (`ImpossibleRoute`).

4.8 Propiedad observada al cierre del desarrollo del modelo

En el CSV piloto, las 210 filas sin contenedor tienen `dias`=0; las 22 con un día tienen al menos un contenedor. Es una propiedad observada del archivo, no una ley universal del servicio.

5 Aplicación, API y uso de inteligencia artificial

5.1 Integración y contratos actuales

La API vigente expone: `GET /scenario`, `GET /containers`, `POST /predictions:predict`, `POST /predictions:explain`, `POST /routes:plan`. La inferencia consume el pipeline ya ajustado; las quejas del escenario se agregan en runtime (no se leen del CSV). Se carga el escenario de ciudad y `arbol-montevideo.joblib` por defecto.

5.2 La aplicación en funcionamiento

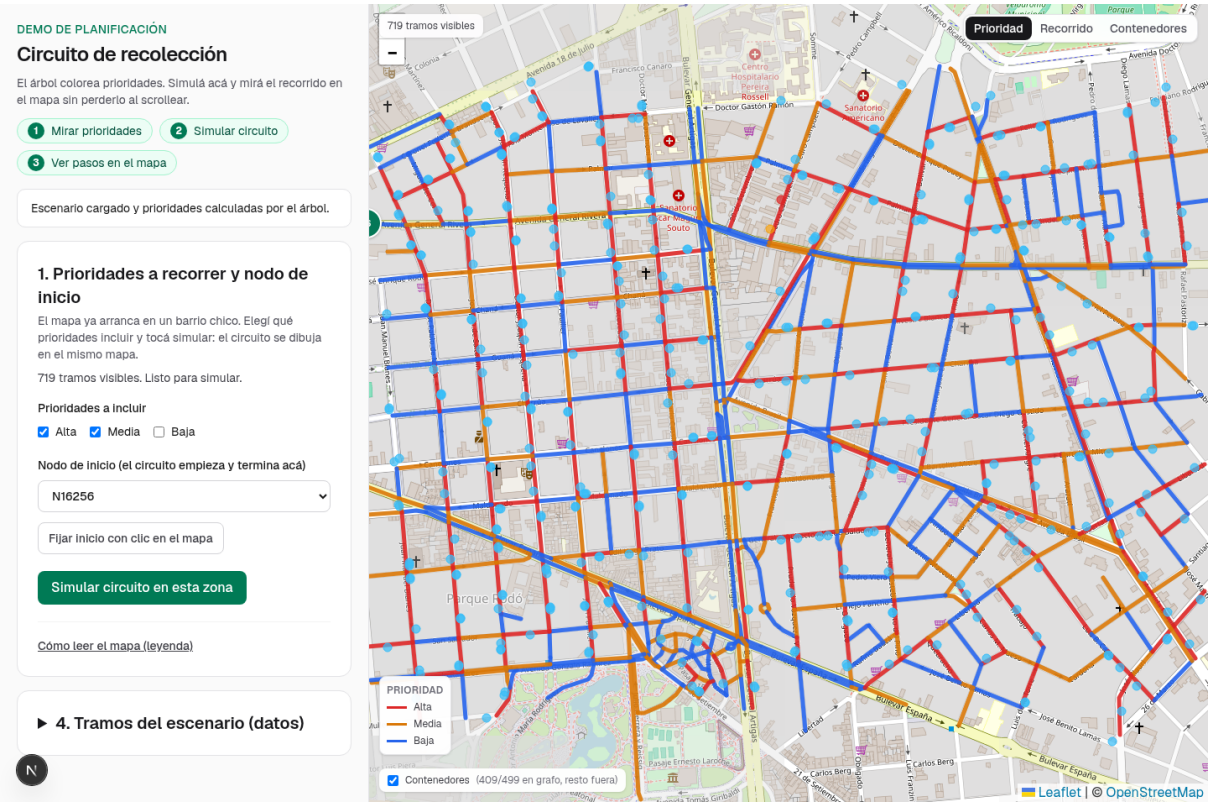


Figura 6: Interfaz actual antes de simular: filtros, nodo de inicio y mapa coloreado por prioridad predicha.

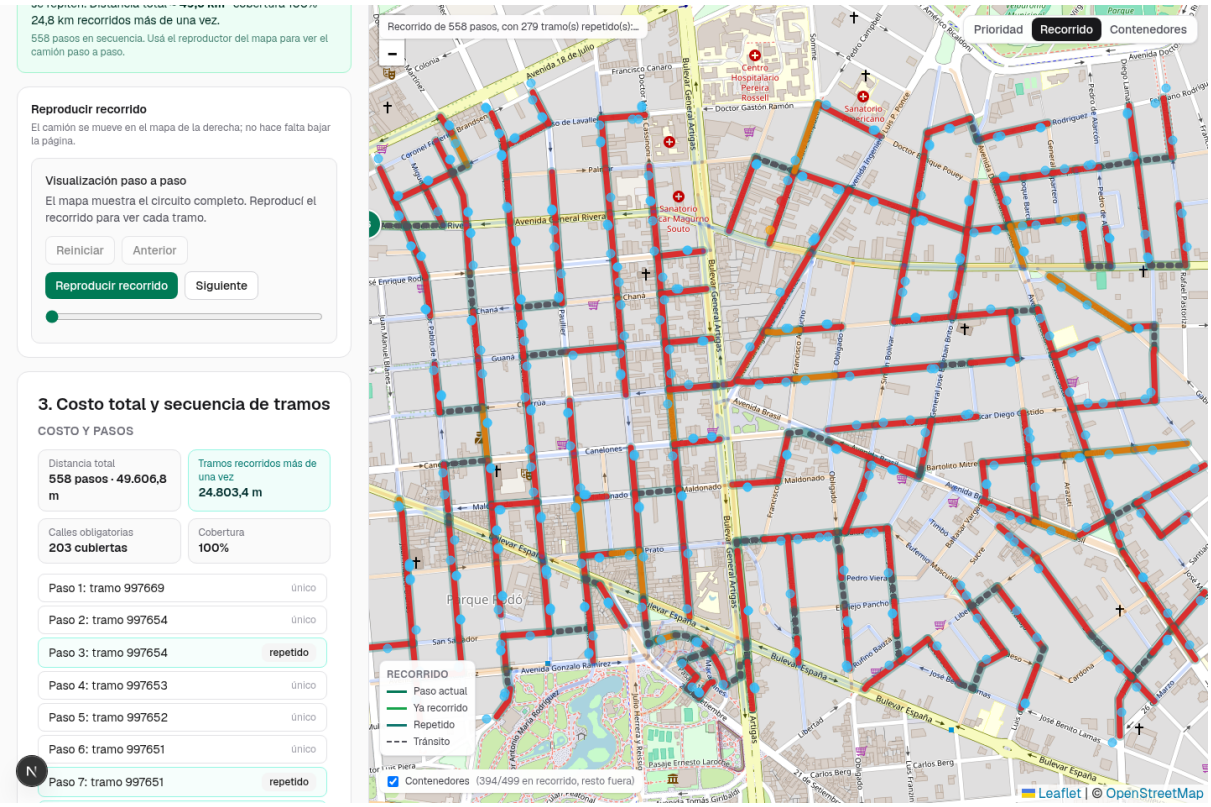


Figura 7: Interfaz actual tras calcular un recorrido: mapa, reproducción, costo y secuencia.

5.3 Declaración de uso de IA

Herramientas: Cursor (agente Composer), GitHub Copilot y asistentes de chat para redacción/documentación.

Dónde sí: documentación y flujo de trabajo; scripts del dataset; tests; parte de la implementación ML/API/frontend; redacción y compilación de este informe.

Dónde no: definición de qué modelos comparar; arquitectura y tecnologías elegidas; comprensión de las técnicas de ML y de grafos; búsqueda y selección del dataset.

Prompts útiles:

> "Revisa las respuestas generadas por el arbol y busca inconsitencias."
>
> "Reescribre esta funcion siguiendo las buenas practicas anunciadas en este link https://refactoring.guru/es/design-patterns/adapter"

Error de la IA y corrección: en una primera formulación, el asistente describió una rama del árbol como "prueba de que el modelo entendió el dominio". Se corrigió a "propiedad observada del CSV": el 1.0 no implica comprensión operativa.

Reflexión: la IA aceleró boilerplate, tests y redacción, pero no sustituye decidir el protocolo de evaluación, justificar el árbol frente a KNN/CV, ni explicar Hierholzer (propio) y el uso de Dijkstra vía NetworkX en la defensa. Detalle adicional en `ai.md`.

6 Grafos: del conjunto de destinos al circuito

6.1 Cómo se conectan el modelo y el grafo

La salida del clasificador es una prioridad por `tramo_id` (`baja`/`media`/`alta`). Esa predicción **no** es el peso de la arista: el peso sigue siendo la longitud en metros. La predicción decide *qué* aristas son obligatorias.

Flujo concreto (código):

1. `POST /predictions:predict` (`backend/src/collection/transport/api_v1.py`) aplica el pipeline `arbol-montevideo.joblib` y devuelve prioridades.
2. El frontend (o el cliente) arma el mapa `priorities` y el filtro `include` (por defecto media y alta).
3. `POST /routes:plan` llama a `PlanRoute` (`backend/src/collection/application/use_cases/plan_route/u` que delega en `NetworkxRoutePlanner` (`backend/src/collection/adapters/graphs/networkx_route_plan`
4. El planner marca como obligatorias las aristas con contenedores cuya prioridad predicha $\in$ `include`; el resto puede usarse solo como tránsito.

Si cambia la predicción (u overrides / `include`), cambia el conjunto obligatorio y, en general, el circuito. Eso es la integración pedida por el enunciado: ML $\rightarrow$ selección de aristas $\rightarrow$ algoritmo de grafos.

La tarea toma los tramos con contenedores cuya prioridad está incluida (por defecto, media y alta) como destinos obligatorios. Los tramos sin contenedores pueden servir de tránsito. La prioridad selecciona qué cubrir; no dicta el orden de visita.

6.2 Qué implementa el planificador

El adaptador `networkx_route_planner.py` construye un multigrafo no dirigido ponderado por longitud. Si los componentes obligatorios están separados, los conecta con caminos mínimos sobre la red completa y un MST auxiliar (heurística rural-postman; no óptimo global). Después, sobre esa red, calcula caminos entre vértices impares con **Dijkstra de NetworkX** (`nx.single_source_dijkstra` / `nx.multi_source_dijkstra`), forma un emparejamiento de peso mínimo con `nx.min_weight_matching`, duplica esos caminos y obtiene el circuito con **Hierholzer propio** (`domain/algorithms/hierholzer.py`).

NetworkX aporta la estructura `MultiGraph`, Dijkstra, el emparejamiento y comprobaciones auxiliares (`is_eulerian`, componentes). No se usa `nx.eulerize` ni `nx.eulerian_circuit`.

```
# networkx_route_planner.py (esquema)
matching = nx.min_weight_matching(complete, weight="weight")
# ... se duplican caminos entre impares ...
circuit = hierholzer(_hierholzer_adjacency(euler_graph), start)
```

El costo reportado es la suma exacta del circuito producido (incluye repeticiones). El grafo se modela no dirigido (simplificación del enunciado).

6.3 Integración y límites prácticos

Endpoint: `POST /routes:plan`. Si no hay destinos o la red no conecta, se devuelve error controlado. El costo no es una predicción del árbol. La demostración debe acotarse a una zona; no se afirma optimización de flota a escala ciudad.

Conclusión

El sistema conecta clasificación de prioridad (árbol) con un circuito tipo cartero chino/rural sobre la red vial de Montevideo. Las métricas del piloto miden reproducción de una regla sintética; el valor del trabajo está en el flujo reproducible datos→modelo→grafo→API→mapa y en declarar sus límites.

Fuentes y referencias verificables

- `docs/dataset-recoleccion.md`, `docs/modelo-comparacion.md`, `docs/comparacion-evaluacion/CONCI`
- `docs/contracts/api-v1.md`, `backend/README.md`, `ai.md`.

– Geofabrik Uruguay OSM; Figshare Montevideo Garbage Containers (DOI arriba).